

Universidade Estadual de Campinas
Instituto de Computação

Introdução ao Processamento Digital de Imagens (MC920 /
MO443)
Professor: Hélio Pedrini

Trabalho 4

Julio Vinicius Amaral Oliveira – 230537

Data de Entrega: 13 de junho de 2025

1 Introdução

O objetivo deste relatório é aplicar 2 algoritmos, o primeiro que deve identificar os contornos dos objetos de uma imagem, além de suas características como: área, perímetro, excentricidade e solidez. O segundo, deve ser capaz de realizar tanto a rotação quando ampliação de uma imagem, utilizando interpolações para tal. As interpolações que vamos trabalhar são: Lagrange, bicúbica, bilinear e pelo vizinho mais próximo.

2 Especificação do Problema

O objetivo principal é desenvolver um programa que: consiga realizar a identificação de do contorno dos objetos além de mostrar suas propriedades e também desenvolver outro que: consiga, utilizando interpolação, realizar a rotação e redimensionamento de imagens.

3 Detecção de contornos

3.1 Bibliotecas utilizadas

Para realização do objetivo, utilizamos algumas bibliotecas como `argparse` para criação das flags, `cv2` para leitura e algumas operações com imagens, `skimage` para obter os ids de cada imagem, propriedades, etc, `numpy` para realizarmos operações matemáticas e por fim o `matplotlib` para gerar os histogramas solicitados

3.2 Descrição do programa

Primeiramente, realizamos a leitura e conversão das imagens usando o `open-cv`, após isso com a ajuda do `threshold.otsu` obtemos o valor de `threshold` e também criamos os labels das regiões. Depois, obtemos as propriedades de cada região com o auxílio da classe `measure` do `skimage` e para cada propriedade imprimimos nos arquivos de output seus valores. O próximo passo, foi detectar os contornos da imagem com a ajuda do `open-cv`, após acharmos seus contornos com a função `cv2.findContours` os desenhamos em um fundo branco com `cv2.drawContours` em vermelho. Após isso, a próxima tarefa foi mostrar os objetos com seus respectivos ids, para isso, bastou utilizar o método `color.label2rgb` do `skimage` e depois precisamos somente escrever os respectivos números acima de cada objetos, para isso achamos o centro e deslocamos ele em x e y considerando o tamanho do texto de forma que esse ficasse no centro do seu respectivo objeto. Por fim, tivemos apenas que percorrer cada objeto e os classificar como pequeno médio ou grande utilizando um laço de repetição simples. Também plotamos o histograma das áreas utilizando funcionalidades simples da biblioteca `matplotlib`.

4 Transformações geométricas

4.1 Bibliotecas utilizadas

Para implementar rotação e redimensionamento de imagens, utilizamos algumas bibliotecas como `argparse` para criação das flags, `cv2` para operações com imagens como leitura, escrita, etc e `numpy` para as operações com matrizes e interpolação.

4.2 Descrição do programa

Primeiramente, realizamos a leitura e conversão das imagens usando o `open-cv`. Depois, criamos quatro rotinas de interpolação que podem ser escolhidas pelo usuário no momento da inicialização do código: vizinho mais próximo, bilinear, bicúbica e Lagrange. Cada uma delas recebe coordenadas (x', y') da imagem original e devolve o valor interpolado utilizando o respectivo método escolhido:

- **Vizinho mais próximo:** simplesmente arredonda (x', y') para o pixel inteiro mais próximo.
- **Bilinear:** calcula a média ponderada dos quatro pixels vizinhos que formam o quadrado ao redor de (x', y') .
- **Bicúbica:** utiliza a função B-spline para realizar a interpolação
- **Lagrange:** aplica polinômios de Lagrange.

Encapsulamos todas as funções numa função `interpolate`, que recebe a imagem, as coordenadas e o método desejado. Essa função então escolhe qual interpolação aplicar com base no que o usuário escolheu. Optamos por não detalhar a fundo as funções, já que sua implementação é apenas a codificação das funções apresentadas no comando do trabalho.

5 Principais desafios

5.1 Detecção de contornos

Os métodos criados para resolução do problema não foram muito desafiadores, se tratando mais de aplicação simples de métodos já existentes em bibliotecas open source. Porém, considerei desafiador a pesquisa para encontrar os métodos que conseguissem resolver os problemas propostos.

5.2 Transformações geométricas

O desafio apresentado envolvia basicamente a implementação das fórmulas apresentadas no enunciado, algumas sendo relativamente simples de implementar como a de vizinhos próximos outras que apresentaram uma complexidade alta. Como a de lagrange, que demandou bastante refatorações até chegar em um

código limpo. Também foi um desafio lidar com as bordas, que durante o desenvolvimento causaram bastantes bugs ao longo do código e que exigiram um manuseio cuidadoso ao longo do código.

6 Resultados

Os resultados obtidos para os respectivos desafios foram

6.1 Detecção de contornos

Para o objeto 2, que possuía mais objetos:

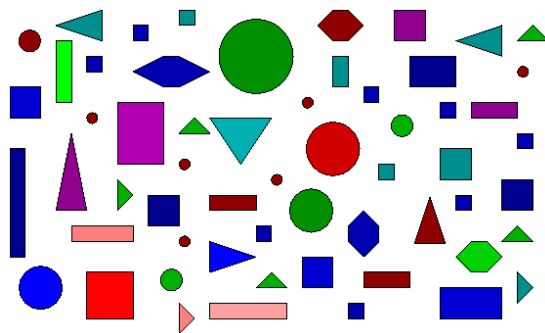
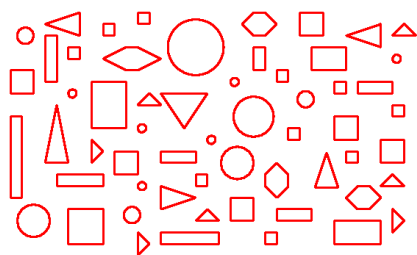
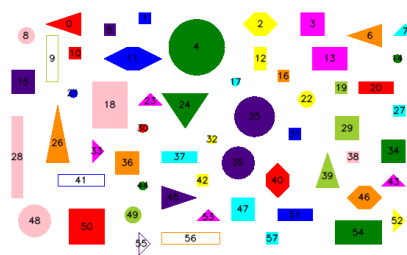


Figura 1: Objeto 2 com diferentes formados

Conseguimos obter seus contornos e seus rótulos:



(a) Contornos da imagem



(b) Rótulos da imagem

Figura 2: Resultados obtidos

Além de seu histograma:

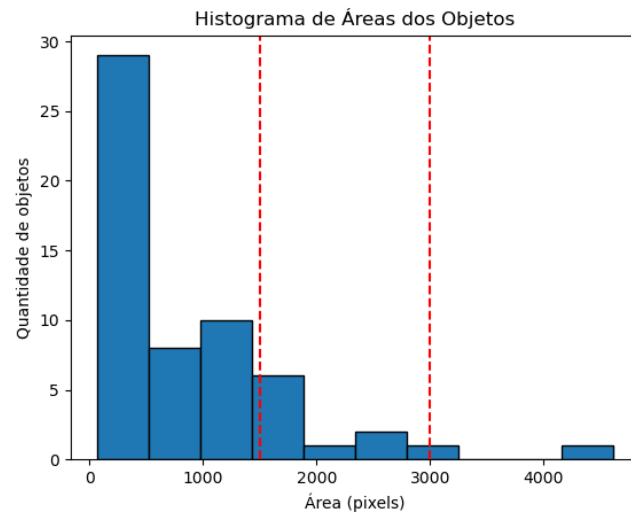


Figura 3: Histograma de áreas

6.2 Transformação

Utilizamos o objeto 1 para os testes:

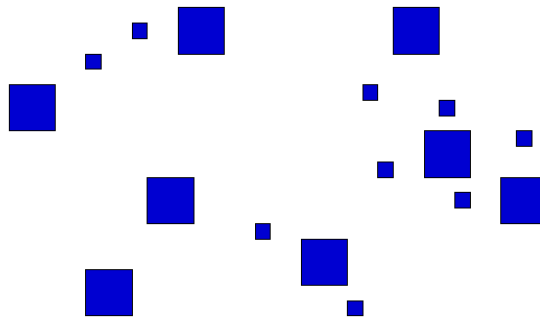
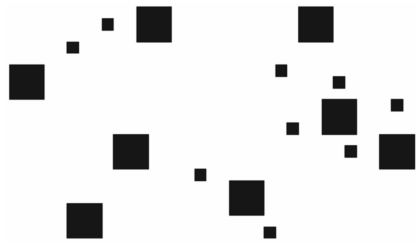


Figura 4: Objeto 1

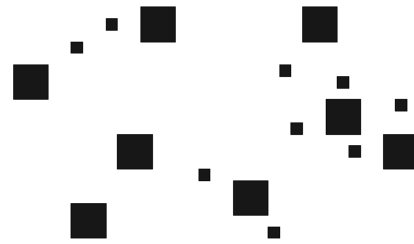
Seguem alguns resultados



Figura 5: Objeto 1 rotacionado em 45 graus (bicúbica)

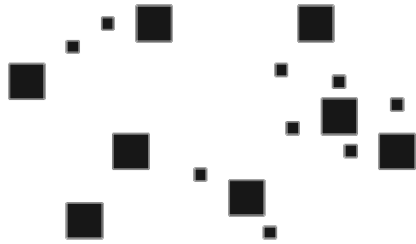


(a) Bicúbica redimensionada para 2x2

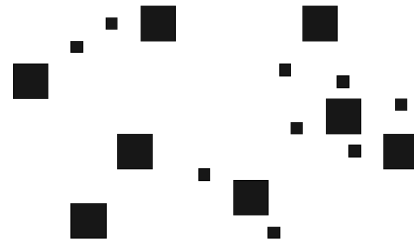


(b) Bilinear redimensionada para 2x2

Figura 6: Interpolações Bicúbica e Bilinear



(a) Lagrange redimensionada para 2x2



(b) Nearest redimensionada para 2x2

Figura 7: Interpolações Lagrange e Nearest Neighbor

7 Conclusão

As implementações dos algoritmos para resolução dos problemas propostos foram bem eficazes, e conforme os resultados mostraram, eles conseguiram atender às solicitações. Como ponto de melhorias, podemos melhorar o desempenho das transformações que utilizam lagrange, pois notamos que o tempo de execução

era bem grande para algumas imagens. Além disso, as imagens que foram rotacionadas, foram cortadas preservando as medidas de largura e altura originais, uma melhoria que poderia ser feita é melhorar isso de forma que a imagem rotacionada não tivesse mais esse corte.